Cloud Computing (CS3081) Mini Project Report

Distilled Command Parser: Local Cloud Deployment and Evaluation

Author: Biram Praveen
Roll Number: 230101014
GitHub Repository: https://github.com/PRAVEEN1404B/cloud-assignment
Commit / Tag: `final`

---

1. System Architecture Overview

The system implements an end-to-end cloud-native machine learning pipeline for a sequence-level distilled command parser deployed on a local k3d Kubernetes cluster.

```
[ Client / Ingress (Port 8080) ]


   [ Traefik Ingress Controller ]


[ command-parser Service (Port 8000) ]


  [ Pod: parser-1 ]       [ Pod: parser-2 ]
  (1 CPU, 1Gi RAM)        (1 CPU, 1Gi RAM)



     [ SeaweedFS S3 Object Storage ]
       - Bucket: datasets (data/v1)
       - Bucket: models   (models/v1, v2)
```

---

2. Task 1: Object Storage and Parallel Teacher Labelling

Teacher Setup & Sharding Strategy

- **Teacher Model:** `phi3.5:3.8b-mini-instruct` via Ollama with greedy sampling (`temperature=0, seed=42`).
- **Prompt:** Defined in `prompts/teacher_prompt.txt` requesting exactly the four keys (`intent`, `date`, `time`, `place`).
- **Sharding:** 3,000 commands partitioned across $K = 4$ parallel workers using modular indexing: `line_index % 4 == SHARD_INDEX`.
- **Fault-Tolerance / Resumability:** Each worker periodically checkpoints output to SeaweedFS S3 (`datasets/teacher/shard-{i}.jsonl`). Upon interruption, the worker queries the S3 object length and resumes from the exact next command without duplicate invocations.

Clean Counts Table

Category	Description	Count
Input Pool	Total commands from `data/commands_pool.jsonl`	3000
(a) Invalid JSON	Unparsable syntax or non-JSON output	8
(b) Wrong Keys	Missing or superfluous dictionary keys	3

Category	Description	Count
(c) Invalid Intent	Intent not in the 8 permitted categories	5
(d) Hallucinated Slots	Slot string not found verbatim in input command	75
Duplicates	Repeated command texts (retaining lowest ID)	44
Kept (Cleaned)	Valid, deduplicated training records	2865
Sum Accounted	$(a)+(b)+(c)+(d)+$duplicates$+$kept	$8 + 3 + 5 + 75 + 44 + 2865 =$ 3000 (asserted)

- **Dataset Checksum (`train.sha256`):** `2b17c88cfd3c5a3fa19b8a4029a0a301fb4b13bbd0349d377a32608b29165d2f`

---

3. Task 2: Training the Student (Sequence-Level Distillation)

Sequence-Level vs. Logit-Based Distillation

- **Sequence-Level Distillation:** Trains the student causal LM using standard cross-entropy loss against the teacher's greedily decoded text output sequences ($\mathcal{Y} = \arg\max P_T(Y|X)$). The student only observes the discrete token output without requiring teacher internal activations.
- **Logit-Based Distillation:** Minimizes Kullback-Leibler (KL) divergence between student logits and teacher softened probability distributions ($\tau > 1$) over the vocabulary at every generation step.
- **Why Logit-Based Distillation is Infeasible Here:**
  1. The teacher (`phi3.5`) and student (`pythia-70m`) use completely distinct tokenizers and incompatible vocabulary spaces (phi3.5 vocab size ~32,064 vs. Pythia vocab size 50,304). Direct logit projection is impossible without an alignment layer.
  2. Ollama serves the teacher as a black-box quantized runtime exposing text generation endpoints rather than dense full-vocabulary logit tensors.
- **Limitation:** Errors made by the teacher (misclassifications or missed slots) become hard ground-truth labels for the student model, propagating systematic errors.

Model Evaluation Table (validation.jsonl, 100 samples)

Metric	Teacher (phi3.5)	Untrained (Pythia-70M)	Student v1 (70M Distilled)	Student v2 (70M Distilled)
JSON Validity	99.0%	0.0%	100.0%	99.0%
Intent Accuracy	78.0%	0.0%	78.0%	77.0%
Date Accuracy	86.0%	0.0%	79.0%	77.0%
Time Accuracy	87.0%	0.0%	89.0%	92.0%
Place Accuracy	83.0%	0.0%	81.0%	89.0%
Full-Frame Acc.	49.0%	0.0%	40.0%	49.0%

---

4. Task 3: Containerised Inference Service

- **Image Size:** `1.31 GB` (under the 2.0 GB limit; CPU-only `torch==2.6.0+cpu` wheel, no CUDA libraries).
- **Weight Decoupling:** Docker image contains zero model weights; weights are fetched at startup via S3 credentials from Kubernetes Secrets.
- **Fault Injection (`FAULT_INJECT=1`):** Replaces model generation with `"not json"`, verifying that strict schema validation triggers HTTP 500 with `{"error": "invalid_model_output"}`.
- **Resource Constrained Test (1 CPU, 1 GiB RAM):**
  - Completed 50 sequential requests successfully.

- **Peak Memory Observed:** `661 MiB of the 1 GiB limit` (`docker stats --no-stream`); 49 of the 50 requests returned HTTP 200, total wall time 80.8 s, p50 1.60 s, p95 1.81 s.

---

5. Task 4: Kubernetes Deployment, Autoscaling & Reliability

- **Ingress & Services:** Exposes `/parse` on port 8080 via Traefik.
- **Zero-Downtime Rolling Update (v1 → v2):**
  - Locust load: 4 users, spawn rate 4, continuous traffic.
  - Failed request fraction during update: `1 / 379 = 0.26%` (one HTTP 502 while the Service briefly had no Ready endpoint; required $\leq 0.5\%$). A further 5 requests returned HTTP 500 because the model emitted invalid JSON, independently of the rollout.
  - Post-rollback verification: `X-Model-Version` header confirms immediate reversion to `v1`.
- **HPA Scaling:** Scales from 1 replica to 2 under load as CPU utilization crosses 70%, and stabilizes back down to 1 after 60s cooldown.
- **Cold-Start Time:**
  - Run 1: `16s`, Run 2: `11s`, Run 3: `11s`
  - **Median Cold-Start:** `11s`.
- **Load Balancing (2 Replicas):**
  - Pod A: `50.0%` (65 requests), Pod B: `50.0%` (65 requests) — each $\geq 30\%$. Counted from the `X-Served-By` header in `results/loadtest_headers.json`.

---

6. Task 5: Subnetting and Network Access Control

Subnet Allocation (10.20.0.0/16)

Subnet	Purpose	Prefix	Usable Range	Broadcast	Usable Hosts
`app`	Application Pods	`/22`	`10.20.0.1` – `10.20.3.254`	`10.20.3.255`	1,022
`public`	Load Balancer	`/24`	`10.20.4.1` – `10.20.4.254`	`10.20.4.255`	254
`data`	Storage & DB	`/25`	`10.20.5.1` – `10.20.5.126`	`10.20.5.127`	126
`management`	Admin / Bastion	`/27`	`10.20.5.129` – `10.20.5.158`	`10.20.5.159`	30

NACL Design & Simulator Results

- Passed all sample packets: **6 / 6 (100.0%)**.
- **Rule Order:** Rule 50 (`deny 198.51.100.0/24`) precedes Rule 110 (`allow 443 0.0.0.0/0`) ensuring malicious traffic is dropped before wildcard allows.
- **Stateless Nature:** Explicit outbound rule (Rule 110: ports 1024-65535) is mandatory to permit reply packets to clients.

---

7. Task 6: CI/CD Pipeline & Quality Gate

- **GitHub Actions Workflow:**
  1. Linting (`ruff check`)
  2. Unit Tests (24 pytest tests passing)
  3. Release Asset Evaluation (`score.py`)
  4. Quality Gate (`score.py --min-validity 95`)
  5. Secret Scanning (`gitleaks`)

- **Successful CI Run URL:** `FILL https://github.com/.../actions/runs/...`
- **Quality Gate Failure Demo URL:** `FILL https://github.com/.../actions/runs/...`
  - Fails with: `QUALITY GATE FAILED: json_validity 0.0 < 95.0`.

---

8. Task 7: Performance, Cost & Review

Load Test Performance Summary

- **Requests Sent:** `131`
- **Failures:** `1 (0.76%)`
- **Throughput:** `1.10 req/s`
- **Latency (p50):** `3600 ms`
- **Latency (p95):** `4700 ms` (SLO $\leq$ 5000 ms satisfied)

Cost Estimation (per 1,000 requests)

Using `data/rate_card.json` (INR 3.50/vCPU-hr, INR 0.45/GiB-hr): - 2 replicas running at 2 vCPU and 1 GiB limit (the limits in `k8s/deployment.yaml`):

$$\text{Hourly Cost} = 2 \times (2 \times 3.50 + 1 \times 0.45) = 2 \times 7.45 = INR14.90/\text{hour}$$

- Requests per hour at `1.10` req/s:

$$\text{Requests/hour} = 1.10 \times 3600 = 3,955 \text{ req/hr}$$

- Cost per 1,000 requests:

$$\text{Cost} = \left(\frac{14.90}{3,955}\right) \times 1000 = INR3.77 \text{ per 1,000 requests}$$

Service Level Objective (SLO) & Error Budget

- **SLO Target:** 99.0% successful requests over a 2-minute load test window.
- **Total Requests in Window:** `131`
- **Allowed Error Budget:** $131 \times (1 - 0.99) = 1.31$ requests.
- **Actual Errors:** `1 request (0.76% error rate, 76% of the budget consumed; 23.7% remains)`.

Component Review Table

Component	Evidence (Measured Number)	Limitation Identified	Proposed Improvement
Data & Storage	3,000 commands labelled by 4 in-cluster workers in 107 min	Teacher labelling is the long pole; S3 download adds to cold start	Run the teacher with vLLM on a GPU, cache the model on a volume
Model Quality	Student v1 full-frame 40.0%, teacher 49.0%	70M model still emits ~1% invalid JSON under load	More distilled data, constrained decoding
Latency & Throughput	p95 = 4700 ms, 1.10 req/s at 4 users	CPU autoregressive decoding; nested VM is ~8x slower than the host	Quantised model, batched inference, more replicas
Resource Utilisation	~650 MiB of 1 GiB, CPU saturated at the 2-core limit	4 users already saturate both replicas	Right-size limits and add replicas on a multi-node cluster
Reliability & Recovery	Replacement Pod Ready 15 s after deletion under load; 2/224 failures (0.89%)	Single-node k3d cannot survive host failure	Multi-node cluster with topology spread constraints

Component	Evidence (Measured Number)	Limitation Identified	Proposed Improvement
Security & Cost	INR 3.77 per 1k requests	Secret is only base64; no TLS on the Ingress	Sealed secrets, TLS, tighter egress rules